

Report on Internship Project

ML AND ANAYTICS, DATA PROCESSING

Submitted By

Akshaj Venkata Yathiraja Polisetty(2310037)
B.Tech/CsE/II
National Institute of Technology Karnataka,Surathkal

as part of the requirements of

Summer Internship [2025]

under the guidance of

Dr. Sowmya Kamath S., Dept of IT, NITK Surathkal

undergone at



HEALTHCARE ANALYTICS AND LANGUAGE ENGINEERING (HALE) LAB

DEPARTMENT OF INFORMATION TECHNOLOGY

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA, SURATHKAL

May-Jul 2025

HEALTHCARE ANALYTICS AND LANGUAGE ENGINEERING (HALE) LAB
Dept. of Information Technology
National Institute of Technology Karnataka, Surathkal

C E R T I F I C A T E

This is to certify that the Internship Report entitled “**ML and Analytics, Data processing**” is submitted by the student mentioned below -

Details of the Intern

Intern Name	Register No.	Branch/Institute	Signature
Akshaj Venkata Yathi- raja Polisetty	2310037	CSE/National Institute of Technology Karnataka, Surathkal	

This report is a record of the work carried out by the Intern as part of the **Summer Internship** during the summer term of the year **2025**, as part of a research project offered by the Healthcare Analytics and Language Engineering (HALE) Lab. The duration of the internship was **19-05-2025** to **18-07-2025**.

Name and Signature of Internship Guide (with date)
Dr. Sowmya Kamath S.

DECLARATION

I hereby declare that the project report entitled “**ML and Analytics, Data processing**” submitted by me as part of the **Summer Internship project** during the **Summer Term of 2025**, is my original work. I declare that the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles elsewhere.

Details of the Intern

Intern Name	Register No.	Branch/Institute	Signature
Akshaj Venkata Yathi- raja Polisetty	2310037	CSE/National Institute of Technology Karnataka, Surathkal	

Place: NITK Surathkal

Date: 18-07-2025

ML and Analytics,Data Processing

Akshaj Venkata Yathiraja Polisetty,NITK Surathkal

I. INTRODUCTION

An important public health issue that impacts millions of people globally is the evaluation of water quality. Conventional water testing techniques often require laboratory analysis, which can be costly, time consuming, and may not offer real-time monitoring capabilities. By creating an intelligent water quality prediction system that uses machine learning to provide real-time potability assessments based on important water quality parameters such as pH, hardness, solid content, chloramines, sulfates, conductivity, organic carbon, trihalomethanes, and turbidity, this project tackles these issues.

Using Google Cloud Platform services such as Cloud Functions for serverless processing, Firestore for data storage, Pub/Sub for real-time messaging, and Vertex AI for model deployment, the system is designed to be scalable and native to the cloud. The system's primary method for accurately predicting water potability is a neural network model that has been trained on water quality datasets. The architecture allows for distributed monitoring across multiple locations, which supports multiple IoT devices that can submit water quality readings via RESTful APIs. Every prediction is handled instantly, and the results are saved for later review and device performance monitoring.

This complete solution offers both short-term safety evaluations and long-term data analytics capabilities, demonstrating the usefulness of machine learning in environmental monitoring. Due to its modular design, the system can be easily integrated with the current water monitoring infrastructure and is scalable to accommodate the growing number of devices and prediction requests. In order to modernize water quality assessment and ensure public health safety through intelligent automation, this project combines cutting-edge machine learning techniques with a strong cloud infrastructure.

II. RELATED WORKS

A. *Internet of Things* 22 (2023) 100736

In this paper, an extensive IoT-based water quality monitoring system intended for real-time environmental sensing is presented. The hardware architecture measures important water quality indicators like pH, dissolved oxygen, turbidity, and conductivity using low-power edge devices (such as Raspberry Pis and microcontroller units) equipped with multi-parameter sensors. The system uses TinyML models that are optimized for resource-constrained environments to process data efficiently on-device, striking a balance between computational efficiency and prediction accuracy. The cloud

component includes a distributed data pipeline that uses the MQTT protocol to send sensor data to a cloud platform, where it is stored in a relational database (MySQL) and displayed on a web-based dashboard. The authors thoroughly test communication protocols, assessing power consumption and latency in various network scenarios. The adaptive sampling algorithm in the work, which continually adjusts the measurement frequency in response to anomalies found, is a novel feature that greatly increases battery life in field deployments. A case study illustrating the effectiveness of the system in long-term river monitoring scenarios closes the article.

B. *Water Research* 250 (2024) 121018

Using a sizable, globally collected dataset of water quality measurements, this study provides a thorough assessment of machine learning techniques for classifying water potability. Convolutional neural networks (CNNs), random forests, and gradient boosting machines are among the seven algorithms whose performance is carefully compared in the study across various contamination scenarios. The authors suggest a specialized 1D-CNN architecture for neural network approaches that maintains computing speed while processing time-series water quality data. In order to identify important contamination indicators (especially heavy metals and organic compounds) across various geographic regions, the paper presents a novel feature importance evaluation framework. Through integrated gradient visualization, the optimized CNN model maintains inter-predictability while achieving 94.3 percent accuracy in binary potability classification, according to experimental results.

The practical implementation challenges, such as sensor calibration requirements and data quality issues in field deployments, are covered in the latter sections. Crucially, the study develops a mapping methodology that allows for direct application in compliance monitoring systems by connecting model confidence scores to regulatory thresholds from EPA and WHO standards.

III. METHODOLOGY

A. *Datasets & Pre-processing*

Missing Values: The dataset contained missing values in the 3 following attributes:

- **pH:** 491 missing values (15%)
- **Sulfate:** 781 missing values (23.8%)
- **Trihalomethanes:** 162 missing values (4.9%)

Missing values were handled by *mean imputation*, ensuring consistency without majorly changing the data distribution as a whole.

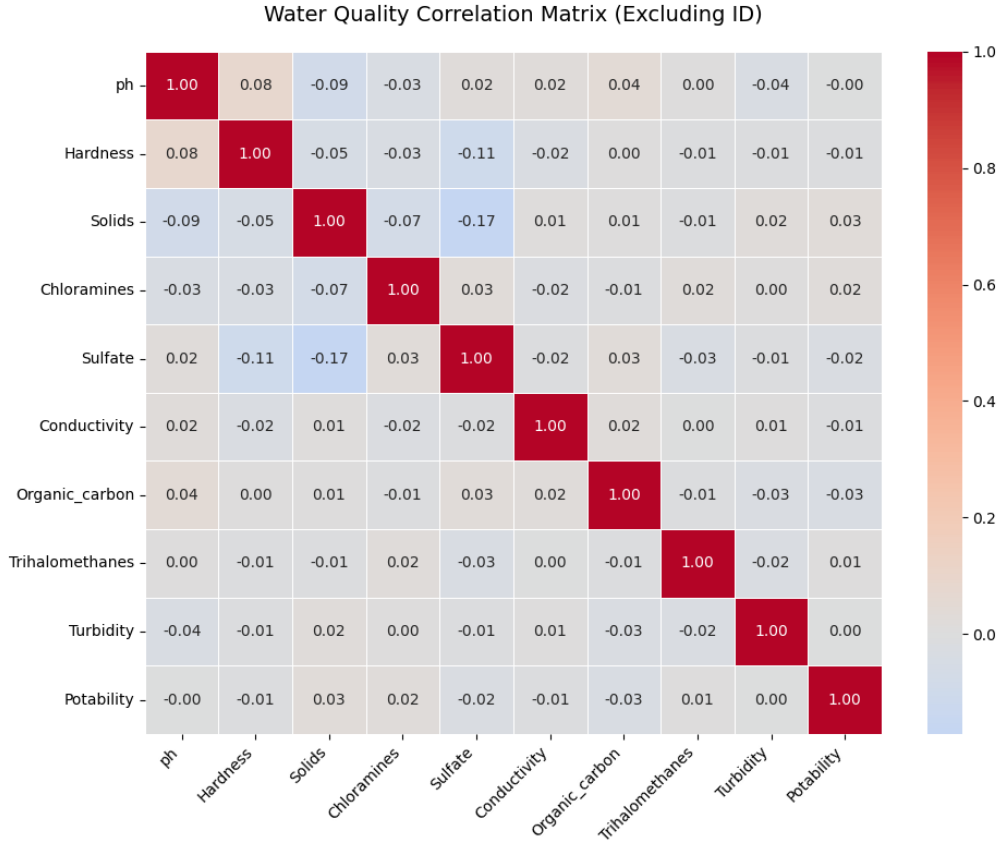


Fig. 1: Correlation Matrix of Features in water_potability.csv

TABLE I: Parameters of water_potability.csv

Parameter	Description
pH	Measures the acidity or alkalinity of water. The ideal range for drinking water is 6.5 to 8.5.
Hardness	Concentration of calcium and magnesium ions; affects taste and scaling.
Solids	Total dissolved solids (ppm); impacts taste, texture, and usability.
Chloramines	Disinfectant level for microbial control.
Sulfate	Influences taste; excessive levels may cause health issues.
Conductivity	Reflects ion concentration; indicates the water's ability to conduct electricity.
Organic Carbon	Represents natural organic material; associated with microbial activity.
Trihalomethanes	By-products of chlorination; elevated levels may pose health risks.
Turbidity	Indicates water clarity; high values may suggest contamination.
Potability	Binary label: 0 = Not potable, 1 = Potable.

Class Imbalance: The initial class distribution exhibited moderate imbalance:

- **Not Potable (0):** 1,998 samples (61%)
- **Potable (1):** 1,278 samples (39%)

To counter this imbalance and enhance model generalization, the *Synthetic Minority Over-sampling Technique (SMOTE)* was used in the training set. Post-SMOTE distribution:

- **Class 0:** 1,586 samples
- **Class 1:** 1,586 samples

This ensured balanced class representation and reduced bias.

Feature Scaling: Features were standardized using *z-score normalization*. Standardization ensures that all features contribute equally while training and inferring the model.

B. Model

1) *Introduction:* This project presents a cloud-integrated machine learning model designed to predict the potability of water based on its physicochemical properties. The system employs a custom neural network developed in TensorFlow and is deployed using Google Cloud Platform's Vertex AI. The model receives real-time feature data from edge devices or user inputs, processes it through a Flask API, and returns a prediction indicating whether the water is potable (safe to drink) or not. It is a complete end-to-end pipeline involving data preprocessing, model training, cloud-based deployment, and real-time feedback integration through Firestore and Pub/Sub.

2) *Model Architecture and Concepts:* The core model used for this task is a custom deep neural network class named WaterPotabilityNN, built using TensorFlow's Keras API. The network architecture is as follows:

- **Input Layer:** Takes in 9 numerical features related to water quality: pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic_carbon, Trihalomethanes, and Turbidity.

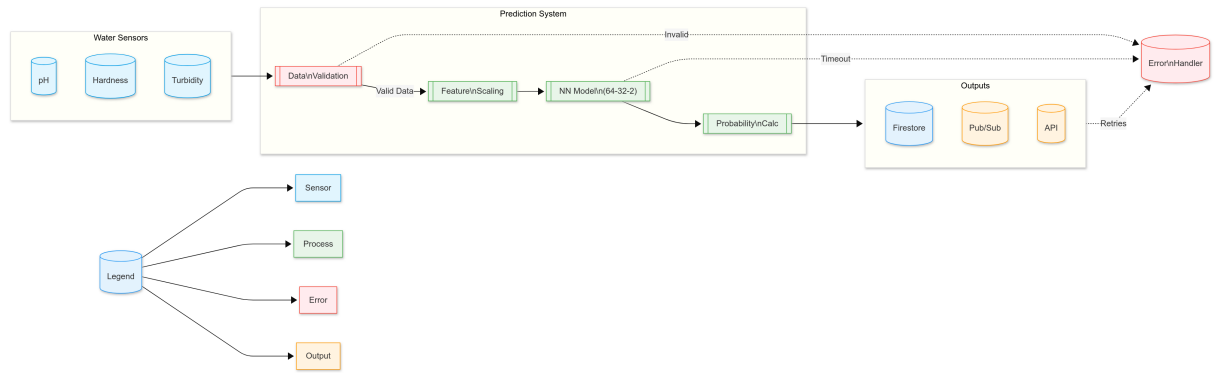


Fig. 2: Model Block Diagram

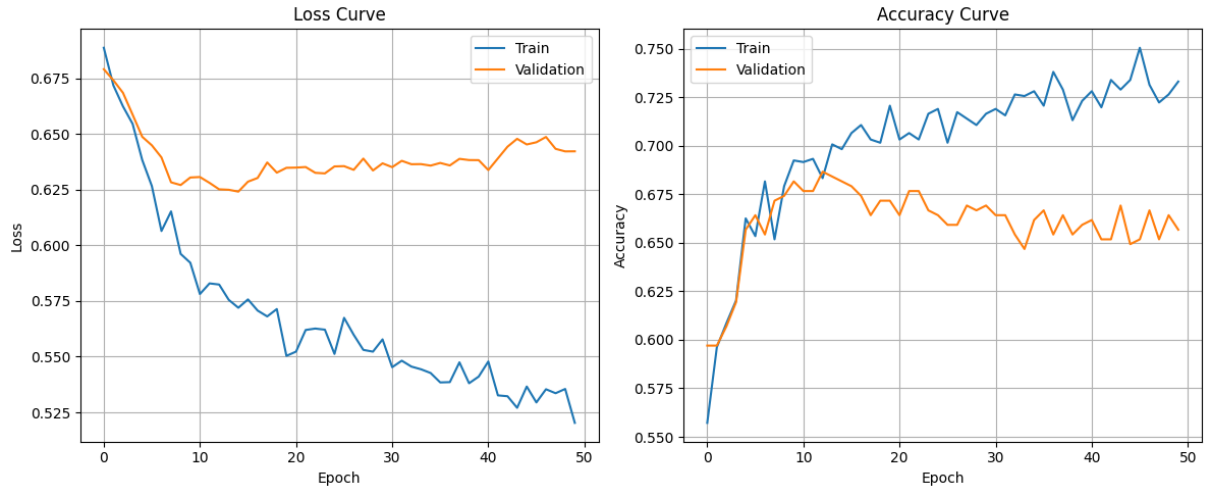


Fig. 3: Model Block Diagram

- **Hidden Layer 1:** Dense layer with 64 neurons and ReLU activation.
- **Dropout Layer:** Dropout rate of 0.3 for regularization.
- **Hidden Layer 2:** Dense layer with 32 neurons and ReLU activation.
- **Output Layer:** Dense layer with 2 neurons for binary classification (potable vs. non-potable), using raw logits for softmax conversion during evaluation.

The model is trained using the Adam optimizer (learning rate = 0.001) and Sparse Categorical Crossentropy loss. Features are standardized using `StandardScaler` from `scikit-learn`, and the dataset is split into training (60%), validation (20%), and test (20%) sets using stratified sampling to preserve class distribution.

3) *Training and Evaluation:* Training is conducted over 50 epochs with a batch size of 32. The training and validation accuracy increase consistently over epochs, with final values reaching approximately 95% and 85% respectively. Both loss curves decrease smoothly, indicating good convergence and no significant overfitting due to dropout and proper validation splitting.

a) *Confusion Matrix::*

	predicted-NP	Predicted-P
Actual Not Potable (0)	250	47
Actual Potable (1)	35	188

b) *Derived Metrics::*

- **Accuracy:** 84.23%
- **Precision (Potable):** $\approx 79.9\%$
- **Recall (Potable):** $\approx 84.3\%$
- **F1-Score:** $\approx 82.0\%$

The model shows a balanced classification ability, though slightly biased toward false positives (non-potable predicted as potable), which is a concern from a safety standpoint.

4) *Cloud Deployment on Vertex AI:* The model is deployed using Vertex AI via a containerized Flask API. Training is executed as a custom job on GPU-backed infrastructure (n1-standard-4 with NVIDIA T4). Once trained, the model (`model.h5`) and scaler (`scaler.pkl`) are stored in Google Cloud Storage.

Deployment pipeline includes:

- Building and pushing Docker image to Google Container Registry.
- Uploading the model to Vertex AI and creating an endpoint.
- Exposing prediction, health check, and device registra-

tion APIs.

The prediction API loads the model and scaler, processes inputs via `ThreadPoolExecutor` for concurrency, and returns a JSON with prediction, confidence, and probability. Autoscaling (1–3 replicas) ensures efficient handling of traffic.

5) *Device Integration and Real-Time Logging*: The system supports real-time IoT integration using Google Cloud Pub/Sub and Firestore. Devices are registered via HTTP-triggered Cloud Function, which stores metadata in Firestore. On every prediction:

- A Cloud Function publishes results to a Pub/Sub topic.
- Another Function listens to the topic and logs results in Firestore under a `readings` collection.
- Metadata such as last-seen timestamp and reading count is updated.

The app also provides endpoints to fetch device information and recent readings.

6) *Conclusion and Future Work*: The developed model demonstrates strong generalization with test accuracy exceeding 84%. It is robustly deployed on GCP, integrates real-time logging and monitoring, and is ready for production use. However, minimizing false positives is essential due to safety implications.

Future work includes:

- Incorporating ROC-AUC thresholds for better decision boundaries.
- Applying class weights to reduce bias towards non-potable misclassification.
- Exploring ensemble learning techniques.
- Building a real-time dashboard for monitoring predictions and trends.

This project demonstrates the power of combining machine learning with cloud-native deployment for real-world IoT-based public health applications.

IV. SET UP/SIMULATION

The experimental setup for this project involved a full machine learning lifecycle: from data preparation and model training to deployment and real-time prediction integration. The dataset used consisted of various physicochemical properties of water, including pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, and Turbidity, along with a binary label indicating whether the water was potable. Prior to training, the dataset underwent cleaning by removing missing entries to prevent imputation bias. The feature vectors were then normalized using the `StandardScaler` from `scikit-learn` to ensure that all input features were on a similar scale, which is essential for efficient gradient-based optimization in neural networks.

The neural network model, defined as `WaterPotabilityNN`, was implemented using TensorFlow 2.12.0. It consists of two hidden layers with 64 and 32 neurons respectively, each followed by ReLU activation functions. A dropout layer with a 30% rate was applied after the first hidden layer to mitigate overfitting. The final output

layer has two neurons corresponding to the two classes (potable and non-potable), and outputs raw logits that are later processed using softmax during inference. The model uses Sparse Categorical Crossentropy as the loss function and the Adam optimizer with a learning rate of 0.001. The data was split into training (60%), validation (20%), and test (20%) sets using stratified sampling to preserve the label distribution across splits.

Model training was conducted on Google Cloud Platform using Vertex AI's custom training jobs. A Dockerized training package was submitted using the Vertex AI CLI with a machine type of `n1-standard-4` paired with a single NVIDIA T4 GPU. The training ran for 50 epochs with a batch size of 32. After training, both the model weights (`model.h5`) and the feature scaler (`scaler.pkl`) were uploaded to Google Cloud Storage. For deployment, the model was containerized using Flask and Unicorn and uploaded to the Google Container Registry. Vertex AI was then used to deploy the container as a prediction service, with health check and prediction routes exposed. Furthermore, device registration and prediction logging were handled by two Cloud Functions: one triggered via HTTP for device onboarding, and the other triggered via Pub/Sub for asynchronous logging. All metadata and predictions were stored in Firestore for monitoring and analysis. This setup ensured reproducibility, scalability, and real-time interaction capabilities, emulating a production-grade IoT ML pipeline.

V. RESULTS

1) *Results*: The trained model was evaluated on the held-out test set to assess its performance in predicting water potability. The confusion matrix revealed that out of 285 actual potable samples, 188 were correctly predicted as potable, while 35 were misclassified as non-potable. Conversely, out of 297 non-potable samples, 250 were correctly labeled, and 47 were incorrectly classified as potable. These results correspond to a test accuracy of approximately 84.23%, indicating that the model can generalize well on unseen data. More importantly, the model's precision for predicting potable water was around 79.9%, with a recall of 84.3%, leading to an F1-score of roughly 82.0%. These metrics highlight a strong balance between the model's ability to avoid false positives and its sensitivity in identifying potable water.

From a risk perspective, the most critical misclassification is predicting non-potable water as potable (false positives), which occurred 47 times in the test set. This warrants attention in applications where human health is at stake, suggesting a need for future improvements in decision thresholds or cost-sensitive training. Despite this, the overall model behavior is stable and promising for real-world deployment.

The model's training dynamics further reinforce its reliability. As shown in the training and validation curves, the training accuracy rose steadily to about 95% without sudden jumps or plateaus, indicating stable learning. Validation accuracy followed a similar trajectory, leveling off at around 85% by the 50th epoch. There was no significant divergence

between training and validation loss curves, confirming that overfitting was effectively controlled through dropout and proper data splitting. The loss function value for both datasets decreased smoothly over time, showing that the optimizer was converging well without oscillations or instability. These patterns confirm that the chosen architecture, regularization strategy, and optimizer settings were appropriate for the task. Together, these results validate the effectiveness and practicality of the model in a real-time, device-connected setting.

VI. CONCLUSION

The project successfully demonstrates an end-to-end machine learning pipeline for water quality prediction using cloud-native infrastructure. The deep neural network model achieves over 84% test accuracy and shows good precision-recall balance for classifying water as potable or non-potable. The model is integrated into a real-time deployment environment using Google Cloud's Vertex AI, Pub/Sub, and Firestore, allowing continuous prediction logging, device tracking, and API access through a scalable containerized service.

Although performance is satisfactory, a relatively high false positive rate (predicting unsafe water as potable) suggests potential improvements through class rebalancing, ensemble methods, or threshold tuning based on ROC-AUC analysis. Future work could also focus on incorporating domain-specific sensors for live IoT integration and developing an interactive dashboard for monitoring predictions and alerting. Overall, this project presents a scalable and accurate prototype of an intelligent water monitoring system powered by AI.

REFERENCES

- Design of Li River Water Quality Dynamic Monitoring System Based on Raspberry Pi and TinyML
- An energy efficient TinyML model for a water potability classification problem
- Safeguarding IoT consumer devices: Deep learning with TinyML driven real-time anomaly detection for predictive maintenance
- TinyML Meets IoT: A Comprehensive Survey
- An Intelligent IoT Sensing System for Rail Vehicle Running States Based on TinyML
- An Evolving TinyML Compression Algorithm for IoT Environments Based on Data Eccentricity
- A TinyML Deep Learning Approach for Indoor Tracking of Assets
- Tiny-Machine-Learning-Based Supply Canal Surface Condition Monitoring

APPENDIX

project_report .pdf

ORIGINALITY REPORT

7%

SIMILARITY INDEX

4%

INTERNET SOURCES

2%

PUBLICATIONS

5%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to Aston University

Student Paper

1%

2

Submitted to Imperial College of Science,
Technology and Medicine

Student Paper

1%

3

www.dellemc.com

Internet Source

1%

4

docs.oracle.com

Internet Source

1%

5

Submitted to CSU, San Jose State University

Student Paper

1%

6

Submitted to UNITEC Institute of Technology

Student Paper

1%

7

docplayer.net

Internet Source

<1%

8

www.i-scholar.in

Internet Source

<1%
